train_unsloth_HICRA_phi_3-5

December 30, 2025

1 Unsloth GRPO Training for Hierarchical Reasoning

This notebook uses Unsloth's optimized kernels with TRL's GRPOTrainer for stable training.

Key features: - ~50% less VRAM usage than standard transformers - vLLM fast inference for generation - HICRA-inspired reward functions for reasoning

[1]:
```
# --- FIX: Disable torch.compile to avoid Dynamo Errors ---
import torch

# Monkeypatch torch.compile to be a no-op decorator/function
def no_op_compile(model=None, *args, **kwargs):
    # 1. Called as torch.compile(model, ...)
    if model is not None and callable(model):
        return model

    # 2. Called as @torch.compile(...) -> returns decorator
    def decorator(func):
        return func
    return decorator

torch.compile = no_op_compile
print(" Disabled torch.compile for stability (Robust Fix).")
```

```
 Disabled torch.compile for stability (Robust Fix).
```

[2]:
```
# Cell 1: Environment Setup.
import os
os.environ["fix_mistral_regex"] = "True"
# os.environ["OMP_NUM_THREADS"] = "1"
os.environ["UNSLOTH_VLLM_STANDBY"] = "1"  # Extra 30% context lengths

# Install dependencies (run this if not already installed)
# !pip install unsloth vllm
# !pip install transformers==4.56.2
# !pip install --no-deps trl==0.22.2
```

[3]:
```
# Set remote HF_TOKEN from local .env
import os
```

```
from dotenv import load_dotenv

load_dotenv()
hf_token = os.getenv('HF_TOKEN')

# ssh -i ~/.ssh/id_ed25519 dataimaginations-heirarchical-reasoning@ssh.hf.space␣
  ↪"echo 'export HF_TOKEN={hf_token}' >> ~/.bashrc"
print(" Token set! Restart remote shell to activate.")
```

```
 Token set! Restart remote shell to activate.
```

[4]:
```
# Cell 2: HuggingFace Login
import os
from huggingface_hub import login
from dotenv import load_dotenv

load_dotenv()
hf_token = os.getenv('HF_TOKEN')

if hf_token:
    login(token=hf_token)
    print(" Logged in with HF_TOKEN")
else:
    login()
    print(" Logged in interactively")
```

```
Note: Environment variable`HF_TOKEN` is set and is the current active token
independently from the token you've just configured.

 Logged in with HF_TOKEN
```

[5]:
```
from unsloth import FastLanguageModel
import torch
# Configuration
max_seq_length = 2048  # Was 1024, now accommodates 512 + 1024 = 1536
lora_rank = 32  # Larger rank = smarter, but slower
print(" Loading model with Unsloth...")
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/Phi-3.5-mini-instruct-bnb-4bit",
    max_seq_length=max_seq_length,
    load_in_4bit=True,
    fast_inference=False,  # requires CUDA toolkit for vLLM
)
# other models: unsloth/phi-4-unsloth-bnb-4bit, unsloth/Phi-3.
  ↪5-mini-instruct-bnb-4bit,
# unsloth/Phi-3-medium-4k-instruct-bnb-4bit, unsloth/
  ↪Phi-3-mini-4k-instruct-bnb-4bit
print(" Attaching LoRA adapters...")
```

```
model = FastLanguageModel.get_peft_model(
    model,
    r=64, # <--- HIGHER RANK (Better learning capacity)
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    lora_alpha=64,
    use_gradient_checkpointing="unsloth",
    random_state=3407,
)
print(" Model loaded successfully!")
```

```
 Unsloth: Will patch your computer to enable 2x faster free finetuning.
 Unsloth Zoo will now patch everything to make training faster!
 Loading model with Unsloth…
==((====))==  Unsloth 2025.12.9: Fast Llama patching. Transformers: 4.57.3.
vLLM: 0.13.0.
   \\   /|    NVIDIA GeForce RTX 4090. Num GPUs = 1. Max memory: 23.484 GB.
Platform: Linux.
O^O/ \_/ \    Torch: 2.9.0+cu128. CUDA: 8.9. CUDA Toolkit: 12.8. Triton: 3.5.0
\        /    Bfloat16 = TRUE. FA [Xformers = 0.0.33.post2. FA2 = False]
 "-____-"     Free license: http://github.com/unslothai/unsloth
Unsloth: Fast downloading is enabled - ignore downloading bars which are red
colored!
 Attaching LoRA adapters…

Unsloth 2025.12.9 patched 32 layers with 32 QKV layers, 32 O layers and 32 MLP
layers.

 Model loaded successfully!
```

Tuning Tips: - If you're getting OOM: Lower NEMOTRON_SAMPLE_SIZE to 1000-2000 - If generations are too long: Lower MAX_ANSWER_TOKENS to 400 - If you want more data: Increase NEMOTRON_SAMPLE_SIZE to 5000+ The filtering keeps ~60-70% of examples typically, so 3000 samples → ~2000 usable examples mixed with your 729 HICRA examples.

[6]:
```
# Cell 4: Load and Combine Datasets
from datasets import load_dataset, Dataset
import json

# === Configuration ===
MAX_PROMPT_TOKENS = 400    # Filter out prompts longer than this
MAX_ANSWER_TOKENS = 600    # Filter out answers longer than this
NEMOTRON_SAMPLE_SIZE = 3000  # How many Nemotron examples to use

# System prompt for reasoning format
SYSTEM_PROMPT = """
You are a mathematical reasoning assistant. Think through problems step by step.
```

```
Respond in the following format:
<think>
...
</think>
<answer>
...
</answer>
"""

def format_prompt(example):
    """Format dataset for GRPO training with chat template."""
    return {
        'prompt': [
            {'role': 'system', 'content': SYSTEM_PROMPT.strip()},
            {'role': 'user', 'content': example['prompt']}
        ],
        'answer': str(example['answer'])
    }

def format_nemotron(example):
    """Convert Nemotron format to our format."""
    messages = example.get('messages', [])

    # Extract user prompt and assistant answer
    user_content = ""
    assistant_content = ""

    for msg in messages:
        if msg['role'] == 'user':
            user_content = msg['content']
        elif msg['role'] == 'assistant':
            assistant_content = msg['content']

    # Get expected answer (fallback to assistant content if not available)
    expected = example.get('expected_answer', '')
    if not expected:
        # Try to extract from assistant's <answer> tags if present
        if '<answer>' in assistant_content and '</answer>' in assistant_content:
            expected = assistant_content.split('<answer>')[-1].split('</
 ↪answer>')[0].strip()
        else:
            expected = assistant_content[-200:] if len(assistant_content) > 200␣
 ↪else assistant_content

    return {
        'prompt': user_content,
        'answer': str(expected)
```

```
    }

def estimate_tokens(text):
    """Rough token estimate (1 token  4 chars for English)."""
    return len(str(text)) // 4

def filter_by_length(example):
    """Filter out examples that are too long."""
    prompt_tokens = estimate_tokens(example['prompt'])
    answer_tokens = estimate_tokens(example['answer'])
    return prompt_tokens <= MAX_PROMPT_TOKENS and answer_tokens <=␣
 ↪MAX_ANSWER_TOKENS

# === 1. Load Your HICRA Synthetic Data ===
print(" Loading HICRA dataset...")
my_dataset = load_dataset(
    "json",
    data_files="reasoning_dataset_v2_train.json",
    split="train"
)
print(f"    Loaded {len(my_dataset)} HICRA examples")

# === 2. Load Nemotron Math Data (Streaming) ===
print(f" Streaming {NEMOTRON_SAMPLE_SIZE} Nemotron math examples...")
try:
    nemotron_stream = load_dataset(
        "nvidia/Nemotron-Post-Training-Dataset-v1",
        split="math",
        streaming=True
    )

    # Take a sample and convert to list
    nemotron_list = []
    for i, example in enumerate(nemotron_stream):
        if i >= NEMOTRON_SAMPLE_SIZE:
            break
        formatted = format_nemotron(example)
        # Only keep if it's not too long
        if filter_by_length(formatted):
            nemotron_list.append(formatted)

        if (i + 1) % 500 == 0:
            print(f"   Processed {i + 1} examples, kept {len(nemotron_list)}...
 ↪")

    nemotron_dataset = Dataset.from_list(nemotron_list)
```

```
    print(f"    Loaded {len(nemotron_dataset)} Nemotron examples (after length␣
 ↪filter)")

except Exception as e:
    print(f"    Could not load Nemotron: {e}")
    print("   Continuing with HICRA data only...")
    nemotron_dataset = None

# === 3. Combine Datasets ===
print(" Combining datasets...")

# Filter HICRA by length too
my_dataset_filtered = my_dataset.filter(filter_by_length)
print(f"   HICRA after filter: {len(my_dataset_filtered)} examples")

if nemotron_dataset and len(nemotron_dataset) > 0:
    from datasets import concatenate_datasets

    # Make sure both have the same columns
    combined_dataset = concatenate_datasets([my_dataset_filtered,␣
 ↪nemotron_dataset])
    print(f"    Combined dataset: {len(combined_dataset)} examples")
else:
    combined_dataset = my_dataset_filtered
    print(f"    Using HICRA only: {len(combined_dataset)} examples")

# === 4. Format for GRPO Training ===
print(" Formatting for GRPO...")
dataset_train = combined_dataset.map(format_prompt)

# Shuffle to mix the datasets
dataset_train = dataset_train.shuffle(seed=42)

# === 5. Load Test Set (HICRA only) ===
dataset_test = load_dataset(
    "json",
    data_files="reasoning_dataset_v2_test.json",
    split="train"
).map(format_prompt)

print(f"\n Final Training Set: {len(dataset_train)} examples")
print(f" Test Set: {len(dataset_test)} examples")
print(f"\nSample prompt format:")
print(dataset_train[0]['prompt'])
```

```
Loading HICRA dataset…
   Loaded 729 HICRA examples
```

```
  Streaming 3000 Nemotron math examples…
Resolving data files:   0%|          | 0/183 [00:00<?, ?it/s]
Resolving data files:   0%|          | 0/159 [00:00<?, ?it/s]
Resolving data files:   0%|          | 0/660 [00:00<?, ?it/s]
Resolving data files:   0%|          | 0/183 [00:00<?, ?it/s]
Resolving data files:   0%|          | 0/159 [00:00<?, ?it/s]
Resolving data files:   0%|          | 0/660 [00:00<?, ?it/s]
    Processed 500 examples, kept 498…
    Processed 1000 examples, kept 998…
    Processed 1500 examples, kept 1498…
    Processed 2000 examples, kept 1998…
    Processed 2500 examples, kept 2498…
    Processed 3000 examples, kept 2997…
      Loaded 2997 Nemotron examples (after length filter)
  Combining datasets…
    HICRA after filter: 729 examples
      Combined dataset: 3726 examples
  Formatting for GRPO…

  Final Training Set: 3726 examples
  Test Set: 36 examples

Sample prompt format:
[{'content': 'You are a mathematical reasoning assistant. Think through problems
step by step.\nRespond in the following
format:\n<think>\n…\n</think>\n<answer>\n…\n</answer>', 'role': 'system'},
{'content': 'Evaluate the integral \\(\\int_0^{2\\pi} \\sqrt{\\sin^2(t)
\\cos^2(t)} \\, dt\\).', 'role': 'user'}]
```

[7]:
```python
# Cell 5: Reward Functions
import re

# Strategic reasoning phrases (from HICRA paper)

STRATEGIC_GRAMS = [
    # Beginning a thought
    "let's analyze", "first we need", "to solve this", "let's assume",

    # Logic Connectors (The most important ones)
    "implies that", "consequently", "therefore", "thus", "because",
    "since", "given that", "conversely", "alternatively",

    # Process Checks (Metacognition)
    "checking the", "verifying", "double check", "but wait", "identifying",
```

```
    "notice that", "recall that", "we can conclude",

    # Mathematical Actions
    "substituting", "calculating", "simplifying", "solving for", "derivative of"
]

def extract_xml_answer(text: str) -> str:
    """Extract answer from <answer> tags."""
    if "<answer>" not in text:
        return text.strip()
    answer = text.split("<answer>")[-1]
    answer = answer.split("</answer>")[0]
    return answer.strip()

def correctness_reward_func(prompts, completions, answer, **kwargs) ->␣
 ↪list[float]:
    """
    Check if the model's answer matches the expected answer.
    Returns 2.0 for correct, 0.0 for incorrect.
    """
    responses = [completion[0]['content'] for completion in completions]
    extracted = [extract_xml_answer(r) for r in responses]

    # Debug output (first item only)
    q = prompts[0][-1]['content'][:100]  # First 100 chars of question
    print(f"---\nQ: {q}...\nExpected: {answer[0]}\nExtracted: {extracted[0][:
 ↪50]}...")

    rewards = []
    for ext, ans in zip(extracted, answer):
        # Check if answer appears in extracted text
        if str(ans).strip() in ext:
            rewards.append(2.0)
        else:
            rewards.append(0.0)
    return rewards

def reasoning_reward_func(completions, **kwargs) -> list[float]:
    """
    HICRA-inspired reward for reasoning structure.
    Gives bonus for using strategic reasoning phrases.
    """
    responses = [completion[0]['content'] for completion in completions]
    rewards = []

    for response in responses:
        score = 0.0
```

```
        response_lower = response.lower()

        # Check for strategic grams
        for gram in STRATEGIC_GRAMS:
            if gram in response_lower:
                score += 0.05

        # Bonus for using reasoning tags
        if "<think>" in response and "</think>" in response:
            score += 0.2
        if "<answer>" in response and "</answer>" in response:
            score += 0.1

        # Cap the reward
        rewards.append(min(score, 0.5))

    return rewards

def format_reward_func(completions, **kwargs) -> list[float]:
    """Reward for correct XML format."""
    pattern = r"<think>.*?</think>\s*<answer>.*?</answer>"
    responses = [completion[0]['content'] for completion in completions]
    return [0.5 if re.search(pattern, r, re.DOTALL) else 0.0 for r in responses]

print(" Reward functions defined")
```

```
 Reward functions defined
```

1.0.1 Chat Template (Save for Base models)

```
# Set Llama 3 chat template (required for GRPO with conversational data)
tokenizer.chat_template = """{% for message in messages %}{% if message['role'] == 'system' %}<
{{ message['content'] }}<|eot_id|>{% elif message['role'] == 'user' %}<|start_header_id|>user<
{{ message['content'] }}<|eot_id|>{% elif message['role'] == 'assistant' %}<|start_header_id|>a
{{ message['content'] }}<|eot_id|>{% endif %}{% endfor %}{% if add_generation_prompt %}<|start_
{% endif %}"""
print(" Chat template set!")
```

Optional: Use More GPU You could also try:

- `num_generations=6` (more diverse rollouts per step)
- Or increase `max_seq_length=1280` in cell 3 if Nemotron answers are very long

1.0.2 How to calculate the Math:

Base Model Cost: A 4-bit model takes ~0.7 GB per billion parameters.Phi-3.5 (3.8B) $\approx$ 2.5 GB.Phi-4 (14B) $\approx$ 10 GB.

Context Cost (The Killer): This is determined by num_generations $\times$ max_completion_length. 16 gens $\times$ 1536 tokens is a lot of data.

The Formula: If nvidia-smi (or equiv for whatever you use) says you are only using 12GB / 24GB, double your num_generations.

This is the safest way to improve performance without changing the model architecture.

[ ]:
```
# Cell 8 (updated)
from trl import GRPOConfig, GRPOTrainer

# --- 2. Training Config for RTX 4090 ---
# Explicitly define these variables to avoid NameError
max_prompt_length = 512
max_completion_length = 1024  # 1024 tokens for reasoning

training_args = GRPOConfig(
    output_dir="phi-3.5-hicra-reasoner",

    # OPTIMIZATION
    learning_rate=5e-6, # Keep low for stability
    lr_scheduler_type="cosine",
    warmup_ratio=0.1,
    optim="paged_adamw_8bit",

    # MEMORY & BATCHING
    per_device_train_batch_size=1,
    gradient_accumulation_steps=1, # Keep small when num_generations is high␣
 ↪(16x)

    # GRPO SPECIFIC (The "Luxury" Settings)
    num_generations=16,       # <--- 16 samples per question for better variance
    max_prompt_length=max_prompt_length,
    max_completion_length=max_completion_length,

    # DURATION
    max_steps=1250, # Start small to test
    save_steps=100,
    logging_steps=1,

    # EFFICIENCY
    fp16=False,
    bf16=True, # 4090 loves Bfloat16
    report_to="tensorboard"
)

print(f" Training configuration set")
print(f"   Prompt: {max_prompt_length} tokens, Completion:␣
 ↪{max_completion_length} tokens")
```

```
Unsloth: We now expect `per_device_train_batch_size` *
```

```
`gradient_accumulation_steps` * `world_size` to be a multiple of
`num_generations`.
We will change the batch size of 1 to the `num_generations` of 16
 Training configuration set
  Prompt: 512 tokens, Completion: 1024 tokens
```

[9]:
```python
# Cell 9: Initialize Trainer
print(" Initializing GRPO Trainer...")

trainer = GRPOTrainer(
    model=model,
    processing_class=tokenizer,
    reward_funcs=[
        correctness_reward_func,
        reasoning_reward_func,
        format_reward_func,
    ],
    args=training_args,
    train_dataset=dataset_train,
)

print(" Trainer initialized!")
```

```
 Initializing GRPO Trainer…
 Trainer initialized!
```

Current settings `nvidia-smi` says: `6861MiB /  12282MiB`

[ ]:
```python
# Cell 8: Run Training!
print(" Starting training...")
print("Note: First ~100 steps may show 0 reward. Be patient!")
print("="*50)

trainer_stats = trainer.train()

print("="*50)
print(" Training complete!")
```

[ ]:
```python
# Cell: Reinitialize for Training (run this before resuming after inference)
"""
import gc
import torch

# Force garbage collection
gc.collect()
torch.cuda.empty_cache()

# Reinitialize accelerator state
```

```
from accelerate.state import AcceleratorState
AcceleratorState._reset_state()
"""
# Then you MUST re-run the trainer initialization cell (Cell 9)
# before resuming training
```

Restart your kernel, run cells 1-9, then run your resume training cell.

[ ]:
```
# Cell 11: Run Training!
print(" Resuming training from checkpoint...")

# Option A: Resume from the latest checkpoint automatically
# trainer.train(resume_from_checkpoint=True)

# Option B: Resume from a specific checkpoint (if you want to go back in time)
trainer.train(resume_from_checkpoint="./phi-3.5-hicra-reasoner/checkpoint-500")

print("="*50)
print(" Training complete!")
```

[ ]:
```
# Cell 9: Save Model
import os
# Option 1: Save locally
output_path = "Phi-4-reasoning-unsloth-HICRA-v1"
model.save_pretrained(output_path)
tokenizer.save_pretrained(output_path)
print(f" Model saved to {output_path}")
# Option 2: Push to HuggingFace Hub (uncomment to use)
repo_name = "DataImaginations/Phi-4-reasoning-HICRA-v1"
hf_token = os.getenv('HF_TOKEN')

# Option 2: Push to HuggingFace Hub (uncomment to use)
# repo_name = "DataImaginations/Llama-1B-Reasoning-v1"
# hf_token = os.getenv('HF_TOKEN')
#
# print(f" Pushing to {repo_name}...")
# model.push_to_hub_merged(
#     repo_name,
#     tokenizer,
#     save_method="merged_16bit",
#     token=hf_token
# )
# print(" Model pushed to Hub!")
```

[ ]:
```
# Cell: Merge LoRA adapters and save for evaluation
from unsloth import FastLanguageModel

# Load the adapter model
```

```
model, tokenizer = FastLanguageModel.from_pretrained(
    "Phi-4-reasoning-unsloth-HICRA-v1",
    max_seq_length=1024,
    load_in_4bit=True,
)

# Merge and save in 16-bit
print(" Merging adapters...")
model.save_pretrained_merged(
    "Phi-4-reasoning-HICRA-v1-merged",  # New path for merged model
    tokenizer,
    save_method="merged_16bit",  # Full precision merged weights
)
print(" Merged model saved!")
```

1.1 Test the Trained Model

[11]:
```
# Cell 11: Test Inference
from unsloth import FastLanguageModel

# Put model in inference mode
FastLanguageModel.for_inference(model)

# Test question
test_question = "A loan is repaid with 20 equal annual payments. The interest␣
 ↪portion of the 16th payment is 400 and the interest portion of the 11th␣
 ↪payment is 600. Find the interest portion of the 1st payment."

messages = [
    {"role": "system", "content": SYSTEM_PROMPT.strip()},
    {"role": "user", "content": test_question}
]

# Tokenize with attention mask
inputs = tokenizer.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_tensors="pt"
).to(model.device)

# Create attention mask (1 for all real tokens)
attention_mask = torch.ones_like(inputs)

# Generate with attention mask
outputs = model.generate(
    input_ids=inputs,
```

```
    attention_mask=attention_mask,  # <-- FIX 1: Add attention mask
    max_new_tokens=1024,
    temperature=0.7,
    do_sample=True,
    pad_token_id=tokenizer.eos_token_id,  # <-- Prevents padding warnings
)

# FIX 2: Decode ONLY the new tokens (skip the prompt)
response = tokenizer.decode(outputs[0][inputs.shape[-1]:],␣
 ↪skip_special_tokens=True)

print("Question:", test_question)
print("\nResponse:")
print(response)
```

```
Question: A loan is repaid with 20 equal annual payments. The interest portion
of the 16th payment is 400 and the interest portion of the 11th payment is 600.
Find the interest portion of the 1st payment.

Response:
<think>
To solve this problem, we need to understand the relationship between the
interest portions of different payments. Since the loan is amortized with equal
payments, the interest portion of each payment will change as the principal
balance decreases over time.
</think>

<answer>
The interest portion of each payment is based on the remaining principal balance
just before each payment is made. Since the payments are equal, we can assume
that the interest portion of each payment decreases linearly as the principal is
paid down.

Given that the interest portion of the 16th payment is 400, and the interest
portion of the 11th payment is 600, we can determine the rate of decrease in the
interest portion per payment.

The difference in the interest portions between the 11th and 16th payments (400
- 600) is a decrease of 200 over 5 payments (from the 11th to the 16th). This
means the interest portion decreases by 40 per payment (200 divided by 5).

Now, to find the interest portion of the 1st payment, we need to add back the
total decrease in the interest portion for the first 10 payments. Since the
interest portion decreases by 40 per payment, the total decrease over the first
10 payments is 40 times 10, which equals 400.

Therefore, the interest portion of the 1st payment is 600 (the interest portion
of the 11th payment) plus 400 (the total decrease over the first 10 payments),
```

giving us a total of 1000.
</answer> <think>
To find the interest portion of the first payment, we need to calculate how much the interest portion decreases each payment as the principal is paid down.
</think>

<answer>
Given that the interest portion of the 11th payment is $600, and the interest portion of the 16th payment is $400, we can determine the rate of decrease in the interest portion per payment.

The difference in the interest portions between the 11th and 16th payments is a decrease of $200 over 5 payments (from the 11th to the 16th). This means the interest portion decreases by $40 per payment (200 divided by 5).

To find the interest portion of the 1st payment, we add back the total decrease in the interest portion for the first 10 payments. Since the interest portion decreases by $40 per payment, the total decrease over the first 10 payments is $40 times 10, which equals $400.

Therefore, the interest portion of the 1st payment is $600 (the interest portion of the 11th payment) plus $400 (the total decrease over the first 10 payments), giving us a total of $1000.
</answer> <think>
The calculation to find the interest portion of the 1st payment involves understanding the rate at0 which the interest portion decreases each payment as the principal balance decreases over time.
</think>

<answer>
With the interest portion of the 11th payment at $600 and the interest portion of the 16th payment at $400, we can calculate the rate of decrease in the interest portion per payment.

The difference in the interest portions between the 11th and 16th payments is a decrease of $200 over 5 payments (from the 11th to the 16th). This means the interest portion decreases by $40 per payment (200 divided by 5).

To find the interest portion of the 1st payment, we add back the total decrease in the interest portion for the first 10 payments. Since the interest portion decreases by $40 per payment, the total decrease over the first 10 payments is $40 times 10, which equals $400.

Therefore, the interest portion of the 1st payment is $600 (the interest portion of the 11th payment) plus $400 (the total decrease over the first 10 payments), giving us a total of $1000.
</answer> <think>
To calculate the interest portion of the first payment, we must understand how

2 Bench for unsloth_HICRA

[ ]:
```python
import lm_eval
from lm_eval.models.huggingface import HFLM

# 1. Load your local model
# If you just saved your model to "llama-1b-reasoning-final", point to that␣
 ↪folder.
print(" Loading model for evaluation...")

# We wrap the model in the Harness's HFLM wrapper
# 'pretrained' can be a local path OR a Hub ID (e.g., "david-barnes/my-model")
llm = HFLM(
    pretrained="llama-8b-reasoning-merged-v1",  # Use the merged model
    batch_size=1,
    trust_remote_code=True,
    dtype="bfloat16"
)

# 2. Define the tasks you want
# These key names correspond to the harness registry.
# Note: "minerva_math" is often split by subject (algebra, etc),
# so we usually run the main "math" group or specific subtasks.
task_list = [
    "aime24",           # AIME 2024
    "minerva_math",     # Minerva Math (covers multiple subjects)
    # "math_500",         # The 'easy' 500 questions from MATH
    "leaderboard_gpqa_main",    # leaderboard_math_hard
]

# 3. Run the Eval
print(f" Running evaluation on: {task_list}...")
results = lm_eval.simple_evaluate(
    model=llm,
    tasks=task_list,
    num_fewshot=0,         # Reasoning models often prefer 0-shot (Instruction)
    limit=None,            # Set to e.g., 50 to test quickly before full run!
    log_samples=True,     # Set True if you want to see exactly what it got wrong
)

# 4. Print a Pretty Table
from lm_eval.utils import make_table
print(make_table(results))

# 5. Save detailed results to JSON (Crucial for your blog!)
import json
with open("llama_8b_unsloth_HICRA_v1_benchmark_results.json", "w") as f:
```

```
    json.dump(results, f, indent=2)
```

3 Soft VRAM clear

[ ]:
```
import torch
import gc

# 1. Delete the Python variables holding the model
# (Wrap in try/except so it doesn't crash if they are already gone)
try:
    del model
    del tokenizer
    del trainer
except NameError:
    print("Variables already deleted or not defined.")

# 2. Python Garbage Collection (Clears CPU RAM)
gc.collect()

# 3. PyTorch Cache Clearing (The most important step for VRAM)
torch.cuda.empty_cache()

# Verify: Print current memory usage
print(f"GPU Memory Allocated: {torch.cuda.memory_allocated() / 1024**3:.2f} GB")
print(f"GPU Memory Reserved:  {torch.cuda.memory_reserved() / 1024**3:.2f} GB")
```